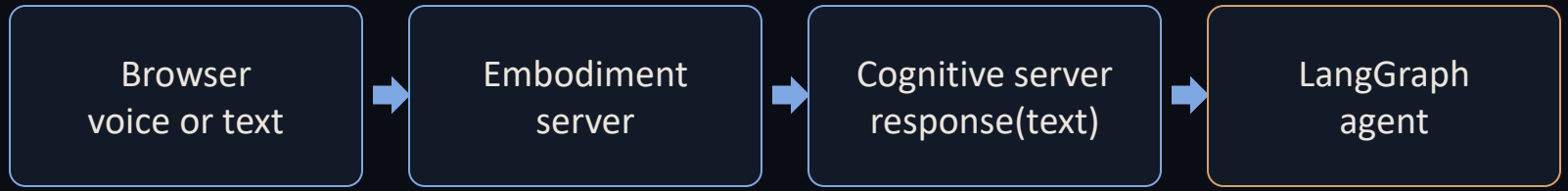
AI-powered Virtual Humans

LLM-Powered Cognitive Architectures

Module 2 - Chapter 03
Integration with the Virtual Human



The Avatar Loop



AgentReply: speech_text, emotion, expression, gesture

The reply travels back: the avatar speaks, moves and changes expression.

What Stays the Same

Frontend

Sends only text.
Plays the reply.
Unchanged.

Contract

response(text)
→ AgentReply

Server routes

/agent
/conversation/clear

Memory is a backend decision: select another agent import.

The Graph Behind response() [1]



InMemorySaver checkpointer · thread_id = "avatar"

respond: prompt template + recent messages → ModelReply

State: Messages and Reply [2]

```
class AvatarState(MessagesState):
    reply: dict
```

messages

The conversation, appended by the reducer.
Only the spoken text is stored.

reply

The latest ModelReply for the avatar: emotion, expression, gesture.

Prompt Templates and Message History [3]

```
prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template(
        "You are a virtual human. Time: {current_time}."),
    MessagesPlaceholder("messages"),
])
```

The system template fills variables. The placeholder keeps message roles.

The respond Node [2]

```
def respond(state: AvatarState):
    answer = structured_client.invoke(
        prompt.format_messages(
            current_time=current_time,
            messages=state["messages"][-MAX_MESSAGES:]))
    return {"messages": [AIMessage(answer.speech_text)],
            "reply": answer.model_dump()}
```

Structured output from Module 1 + rolling buffer from Chapter 02.

The Thread Lives in the Backend [1]

```
THREAD_ID = "avatar"
config = {"configurable": {"thread_id": THREAD_ID}}

def response(text: str) -> AgentReply:
    result = graph.invoke({"messages": [HumanMessage(text)]}, config)
    return AgentReply(**result["reply"])
```

The browser sends only text; every request continues the same thread.

Clear Conversation [1]




```
def clear_conversation():
    checkpointer.delete_thread(THREAD_ID)
```

The next message starts an empty conversation.

Together: Clear Memory through the API

```python
@app.post("/conversation/clear")
def clear_conversation():
    selected_agent = sys.modules[response.__module__]
    clear_agent_memory = getattr(
        selected_agent, "clear_conversation", None)
    if callable(clear_agent_memory):
        clear_agent_memory()
    return {"status": "ok"}
```

Guided exercise: agent hook, server route and browser test

Always Answer

Model error or timeout

A short apology and a shrug.

No structured reply

The same fallback reply.

History in order

Human and AI messages still alternate.

A failed call must not break the avatar or the saved conversation.

Try It on the Avatar

Select the agent

In vh_cognitive_server.py keep one import:
agents.Module_2.Chapter_03
.langgraph_avatar_agent

Test the memory

Tell your name and favourite colour.
Ask for both.
Clear the conversation and ask again.

Save, then restart the cognitive server in the avatar Settings.

Your Turn

Persona

Change the system instruction: a museum guide with its own name.

Buffer

Set MAX_MESSAGES = 4. When does the avatar forget?

Optional

Return a different gesture when greeting and when saying goodbye.

Edit the agent file, save and restart the cognitive server each time.

Module 2 Recap [1] [2]

Chapter 01

Graphs: state, nodes, edges.

Chapter 02

Messages, reducers, checkpointer and threads.

Chapter 03

The avatar remembers the conversation.

Final lab: tools, coding agents and a weather presenter.

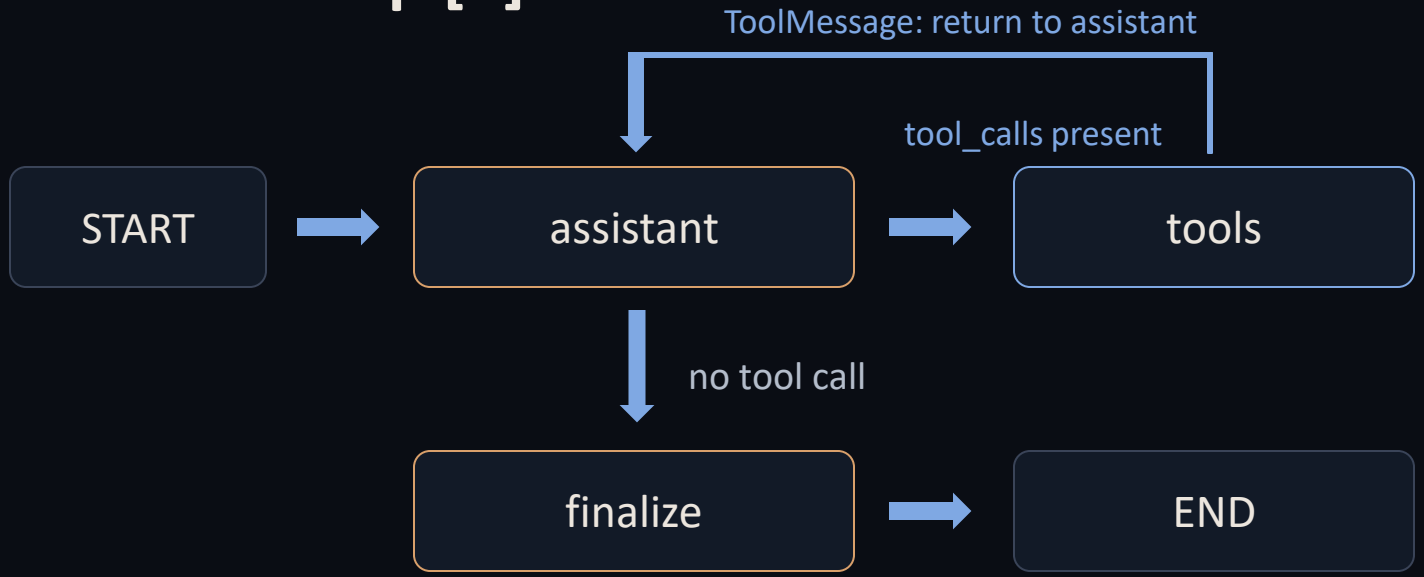
Tools: Data Outside the Model [4]

A tool is a Python function with a name, description and input schema.

```
@tool
def get_weather(city: str, country: str) -> dict:
    """Get current weather for a city and country code."""
    # Python calls OpenWeather and returns real observations.
```

The model requests a call. Your program executes it and returns the result.

The Tool Loop [4]



Repeat after tool execution; finalize produces ModelReply for the avatar.

Wiring the Tool Graph [4]

```
tool_client = client.bind_tools([get_weather])
builder.add_node("tools", ToolNode([get_weather]))
builder.add_conditional_edges(
    "assistant", tools_condition,
    {"tools": "tools", "__end__": "finalize"})
builder.add_edge("tools", "assistant")
builder.add_edge("finalize", END)
```

assistant returns the raw AIMessage, including its tool_calls.

OpenWeather: Current Conditions [7] [8]

1. Resolve city and country to latitude and longitude.

2. Request current weather using those coordinates.

3. Return location, temperature, conditions and observation time.

Use units=metric. Keep the API key in OPENWEATHER_API_KEY.

The Whiteboard API

The avatar can speak and display an image in the same reply.

```
whiteboard = {
    "type": "image",
    "image_url": uploaded_image_url
}
return AgentReply(**result["reply"], whiteboard=whiteboard)
```

null keeps the image. {"type": "clear"} clears the board.

Demo: Weather on the Whiteboard [9]

What is the weather in Rome, Italy?

Download the condition icon selected by the API response.

Upload the image to the embodiment server, then use its local URL.

The avatar reports the observation while the board shows the condition.

Coding Agents [5] [6]

Claude Code and Codex work with the files in a project.

They inspect code, propose edits, run commands and use results to revise.

Give a goal, file paths and acceptance checks. Review the changes and run them.

Codex: Installation and First Task [5] [10]

1. Open the official quickstart and install the desktop app.

2. Sign in with ChatGPT and open your local course project.

3. Ask it to explain the selected avatar agent before editing.

Check account access before class. API-key billing is separate.

Exercise: Memory, Animation and Weather

Start from the memory avatar and add a current-weather tool.

Connect memory, animated ModelReply and Clear conversation.

Show the observed condition on the whiteboard and explain the tool graph.

Exercise brief and suggested Codex prompt

A Useful Prompt for Codex

```
Read the avatar agent and the whiteboard API contract.
Explain how memory and ModelReply currently work.
Add an OpenWeather current-weather tool and a tool loop.
Keep response(text) and clear_conversation().
Download and upload the API condition icon for the board.
Read OPENWEATHER_API_KEY from the environment.
Use mocked HTTP for tests; show the diff and explain it.
```

Provide the full brief. Ask for a plan, then implement and inspect each step.

Weather Agent: Acceptance Checks

Rome, IT: real observation, units and a matching board image.

What about Milan? Uses the conversation context.

Missing city or API failure: asks or explains; no invented weather.

Clear conversation: previous details disappear. Keys stay local.

References

[1] LangGraph persistence

Checkpoints, threads and deleting a thread

[2] ChatOllama structured output

with_structured_output and its methods

[3] Chat prompt templates

System templates and message placeholders

[4] LangGraph tool loop

Model decisions, tools and tool results

[5] Codex quickstart

Desktop installation and ChatGPT sign-in

[6] Claude Code

Coding agent overview

[7] OpenWeather current weather

Current conditions and metric units

[8] OpenWeather geocoding

City names to coordinates

References (continued)

[9] OpenWeather icons

Weather codes and condition icons

[10] Codex plans

Availability and included usage